Australian National University

Lab Five

Design and Implementation of a Digital Door Access System

Overview

This Lab will give you the opportunity to code and implement a complete and complex design, that of a pin-code door access system. A presentation of some aspects of this system will have been given to you in the lectures and/or tutorials and should be mostly included in these notes.

This design is that of a so-called composite state machine (or multi-component system), in which a main machine (the controller) runs a top-level control logic to selectively activate one or several subsystems. These systems can, in themselves, be state machines.

As you undertake this lab exercise, the instructions will lead you through all of the main steps in state machine design. You will need to become comfortable with this process as you will be asked to design a complex system by yourself as part of your assignment.

Important: although it has been designed to be completed within the allocated 3 hours, this lab is quite long. It is strongly recommended that you read this document before coming to the class so that you can focus on any practical issues that may arise on the day.

Australian National University

Table of Contents

Learning Outcomes 2
Description of the system 2
1. The keypad 4
2. The Keypad Processor 6
3. The LED processor *[2 marks]* 7
4. The Memory 8
5. The PIN Comparator *[7 marks]* 9
6. The Door Switch controller 11
7. The hardware reset button 12
8. Putting it all together. The FSM controller *[11 marks]* 12
Appendix 1: using a Basys3 board for this lab 13

Learning Outcomes

- To gain exposure to the complete design and implementation process for a complex state machine.
- To combine all of the key knowledge from the first half of the course.
- To understand the expected level of proficiency and practical skills required for the upcoming assignment.

To complete this lab, you will require access to Vivado and the development Board (Zybo Z7 or Basys 3). Copies of the reference manuals are provided in Canvas.

Description of the system

The aim of the design we approach in this lab is to create a pin-code door access system. Given a very simple keypad with two keys (key 0 and key 1), the user will enter a pin, which is compared with a list of valid pin codes held in the system's memory. If the pin is found amongst the valid codes, access is granted and the door may be opened. Once the door is shut again, the system will wait for a new pin input. If the pin entered by the user is not valid, access will not be granted; the system will reset and wait for a new input.

 © ANU 2025

Further specifications (considering the Zybo Z7 board as the underlying hardware):

- The two buttons representing the 0 and 1 keys could be the two rightmost buttons. A *reset* may also be useful (*left* button), for example to cancel inputting a pin if a mistake has been made. Buttons must be debounced where required.
- As each keystroke is entered, the LEDs should light up indicating how many keystrokes have been entered.
- If access is granted, a particular LED combination should indicate this.

The door could be represented by one of the sliding switches. As the door is unlocked by the system, it must be opened (switch up) and closed again (switch back down) for the system to be reset.

Refer to Figure 1 for a block description of the system:

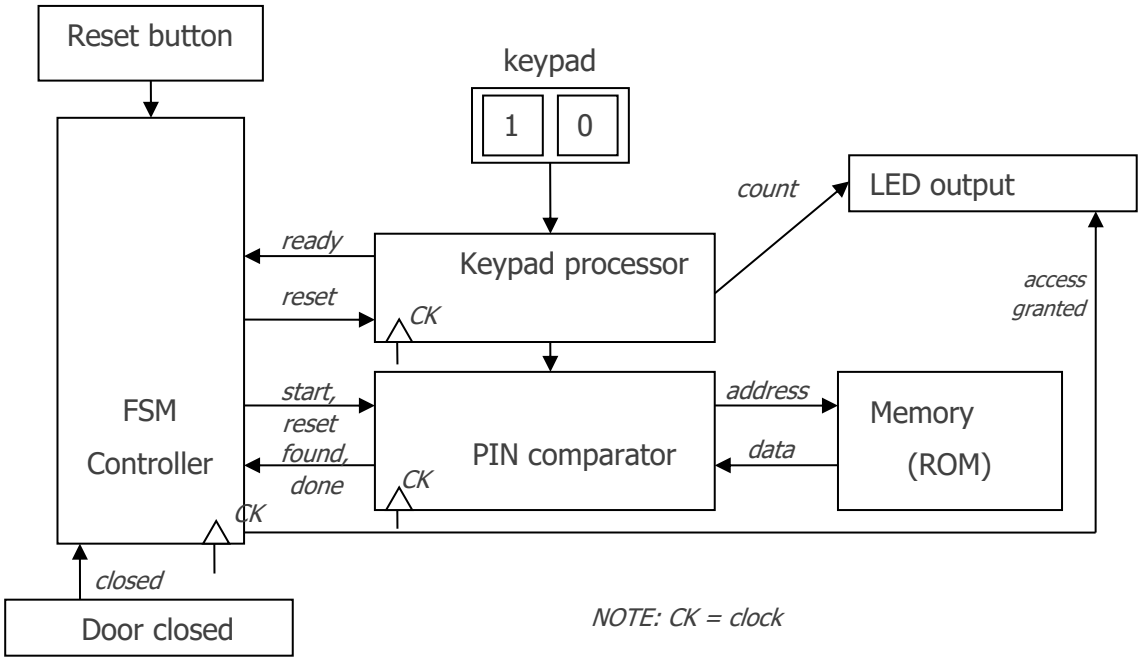

Figure 1: Block diagram description of the door access system

Together with your lab partner, you should *[20 marks]*:

1. Follow the descriptions of the sub-components (one in each subsection), and, following any suggestions provided, ensure that each one is ready to implement. Sometimes the Verilog code will be provided (however, ***it is expected that you can understand any code that is provided to you***), while other times you will have to write it yourselves,

 © ANU 2026

although a module description will be given so that your code can have the right input/output names and be simulated using the test benches provided. Test your designs individually before assembling them together.

2. Write Verilog code for the controller (the code will bring all of the sub-components together) and test bench the whole system
3. Implement the system in hardware and test its operation
4. Show your work results to your demonstrator.

You have the full 3-hour lab to complete this. ***Don't waste time*** as it is a rather long exercise. A functioning hardware implementation will be set up for you to refer to if you have trouble understanding the intended operation of the system. Call on your demonstrator if you are stuck.

1. The keypad

The keypad's role is to transform the pressure of a button into a readable signal. We already know about the issue of push button bounce from the previous labs and you have already designed a debouncer, so the code for that will be provided (see Code 1) in order to save some time. This debouncer has been made slightly more sophisticated because it can

- Reject an event when the simultaneous pressure of the keys 0 and 1 occurs
- Give off a pulse of 1 clock cycle time in length when the button is pressed, regardless of how long the event lasts for (we have called this *single pulse on transition*)

Figure 2 provides a for a debouncer with switch exclusivity.

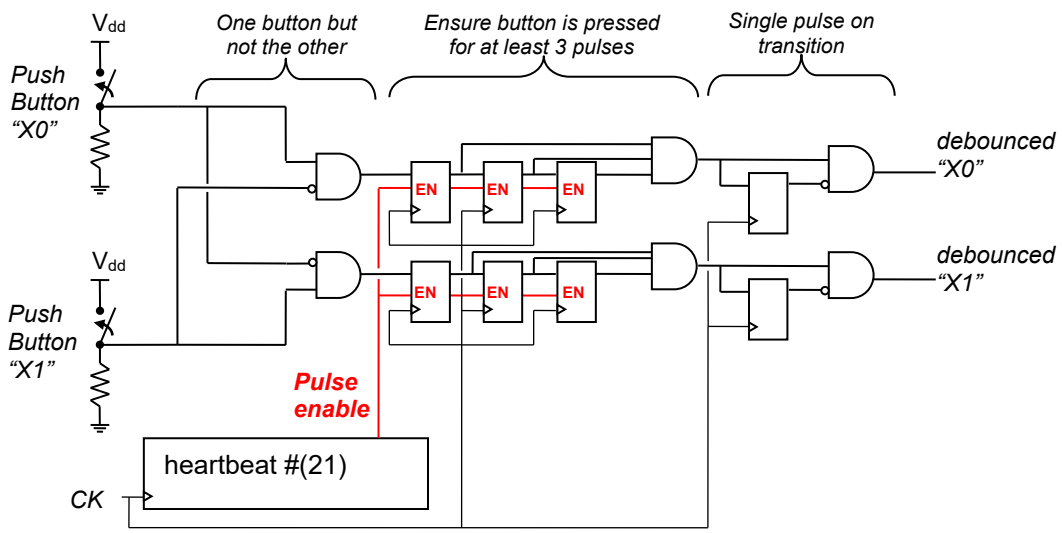

Figure 2: Block diagram of the debouncer with switch exclusivity

 © ANU 2026

This is far from a perfect solution (if you think hard enough you can probably see how a malicious user could press the buttons in a certain order to create an unwanted output) but in order not to make this first block too complicated we will deem this solution acceptable. Below is the Verilog for this application. Spend some time understanding how the code corresponds to the hardware design idea shown in Figure 2.

```
module doubledebounce (input wire X0,
input wire X1,
input wire sysclk,
input wire reset,
output wire X0_deb,
output wire X1_deb     //X0 represents the input for the "0" key, X1 for the "1" key
);

wire pulse;

// heartbeat generator instantiation
heartbeat #(21) hb(.sysclk(sysclk),.reset(reset),.pulse(pulse));

//debouncer variables
reg [2:0] ff0, ff1;   // debouncer flipflop chains for signal 0 and 1
reg ff0_out, ff1_out; // flipflops for single pulse on transition


always @(posedge sysclk) begin    //events which happen at every clock cycle
   ff0_out<=ff0[0]&ff0[1]&ff0[2];
   ff1_out<=ff1[0]&ff1[1]&ff1[2];
   if(reset) begin
      ff0<=3'b0;
      ff1<=3'b0;
   end
   else if(pulse) begin //debouncer section transitions are enabled by pulse
     ff0[0]<=X0&(~X1);
     ff0[1]<=ff0[0];
     ff0[2]<=ff0[1];

     ff1[0]<=X1&(~X0);
     ff1[1]<=ff1[0];
     ff1[2]<=ff1[1];
  end
end


//output logic
assign  X0_deb=(ff0[0]&ff0[1]&ff0[2])&(~ff0_out);
assign  X1_deb=(ff1[0]&ff1[1]&ff1[2])&(~ff1_out);

endmodule
```

Verilog code for the debouncer with switch exclusivity (doubledebounce.v)

© ANU 2026

2. The Keypad Processor

The role of the keypad processor is to detect that a button has been pressed and increase the count (which will be sent to the LED processor). Also the entered key value should be stored in a register and the system should send a *ready* signal to the controller when 8 digits have been entered. When the controller sends a *reset* signal, the count is reset to 0.

For this subcomponent it is easy enough to write a state diagram without too much thinking. The system has states "count" and "ready" (as well as a pin register which is filled as the count increases). Look at Figure 3 to see how the inputs (*reset*, and keys *X0, X1*) determine the state transitions. The names of the states should be intuitive. The pin register is not shown explicitly in the graph, but it is easy enough to understand that it will contain the code digits entered until that point, and that it will be reset to zero if a reset event occurs.

The Verilog code for the keypad processor is provided blow. Take a moment to study it and understand it before you move on to the next section.

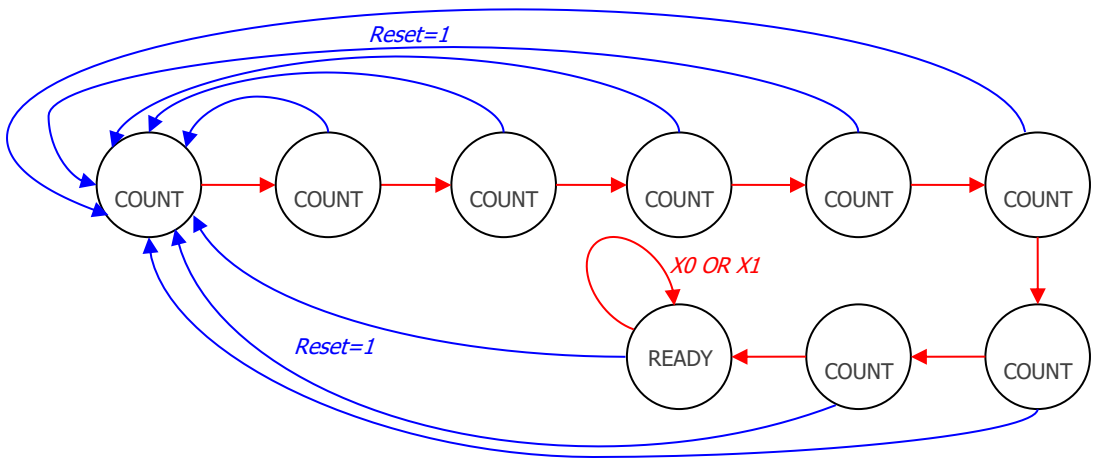

Figure 3: State diagram of the keypad processor

© ANU 2026

```
module keyproc(input wire X0,
input wire X1,
input wire reset,
input wire sysclk,
output wire ready,
output reg [3:0] count,
output reg [7:0] pin_no);

always @(posedge sysclk) begin
if (reset) begin
        count<=4'b0000;
        pin_no<=8'h00;
end
else if ((X0||X1)&&(~ready)) begin
         if (X0)
                pin_no<={pin_no[6:0], 1'b0}; //this syntax implements a shift register
         else                                //make sure you understand it
                pin_no<={pin_no[6:0], 1'b1};
                count<=count+4'd1;
         end
end

assign ready=count[3];
endmodule
```

Verilog code for the keypad processor (keyproc.v)

3. The LED processor *[2 marks]*

This is a very simple module and it is up to you to code it. It is a ***purely combinational block*** (effectively, a decoder) which achieves the following:

- If the overall system is in the *access granted* state, then signal `acc_grant` will be asserted and the "access granted" LED pattern should be displayed.
- Otherwise, a number of LEDs should be lit to help the user keep tabs on the number of pin digits entered

Choose a LED patterns that work for you. The Zybo board only has 4 LEDs so you will need to plan a little to ensure your patterns are user-friendly.

Write the code for this, given the following module structure template (Code 3). A test bench is provided if you need (*TB_LEDproc.v*).

```
module LEDproc(
input wire [3:0] count,
input wire acc_grant,
output reg [3:0] LEDs); //size of bus = number of available LEDs on your board

always @(*)begin

// you will need to write something in here

end

endmodule
```

Module template for the LED processor (LEDproc.v)

© ANU 2026

TIPS: Remember to use blocking or nonblocking assignments as appropriate.

As you write your code consider carefully whether there should be a hierarchy between the behaviours listed above (i.e., count pattern, access pattern).

The output to the LEDs is modelled as a multi-bit word with "1" meaning "on" and "0" meaning "off".

Show your design / simulation result to the tutor when ready. This is a simple task, so you can move on if the tutor is not readily available.

4. The Memory

The memory we intend to use in this setting is a fixed collection of valid PIN codes. In that regard, it can also be coded as a *combinational circuit*. The system is presented with an input (address) and returns the value of the stored data. The history of the inputs does not affect the outputs. The code is provided and is shown below (Code 4). Take enough time to understand how it operates before you move on to the next section.

```
module PIN_rom (input wire [3:0] addr,
output reg [7:0] data);

always @(*) begin
case (addr)
4'h0: data = 8'b00100000;
4'h1: data = 8'b00000010;
4'h2: data = 8'b00000100;
4'h3: data = 8'b00001000;
4'h4: data = 8'b00010000;
4'h5: data = 8'b01111100;
4'h6: data = 8'b01000000;
4'h7: data = 8'b10000000;
4'h8: data = 8'b10000001;
4'h9: data = 8'b10000010;
4'ha: data = 8'b10000100;
4'hb: data = 8'b10001000;
4'hc: data = 8'b10010000;
4'hd: data = 8'b10100000;
4'he: data = 8'b11000000;
4'hf: data = 8'b00000001;
default: data = 8'b00000000; //should not occur
     endcase
end
endmodule
```

Verilog code for the ROM containing the valid PIN codes (PIN_rom.v)

 © ANU 2026

5. The PIN Comparator *[7 marks]*

This is your second hardest task for the day (the hardest will be putting it all together at the end). The PIN Comparator is a subcomponent which checks, upon receipt of a *start* signal, all of the memory addresses (16) of the ROM discussed in the previous section and delivers a *found* signal to the FSM if a matching PIN is found. Once the search is complete, a *done* flag bit is also delivered to the controller (this can be used to determine if the PIN check returned no valid matches).

If we consider the states WAIT (waiting for *start* input), CHECK (PIN checking in progress) and DONE (search complete), the state diagram (Figure 4) for this system is quite simple. The CHECK state is a little bit of a conceptual stretch as there could be several sequential operations (counting of addresses) associated with checking the PIN against the memory values (so CHECK could in fact represent a chain of states. Still, since such a chain would have no branches, this shorter description can be deemed valid).

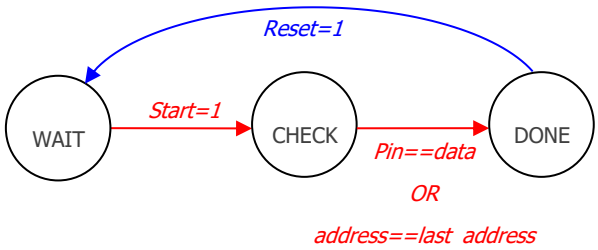

Figure 4: State diagram for the PIN Comparator

It is also intuitive that with such a state representation the system is a simple Mealy machine, since the output (*done*, *found*) depends on the state and on the input (in particular on the equality pin_no==data), as the following tables shows (Table 1). Remember that the *x* indicates a "don't care" entry.

Note that the state encodings have been chosen so that certain bits of the state directly map to outputs. This can reduce the complexity of the output logic. State[1] has been nicknamed chk_enable because it may be used to enable processes associated with the "CHECK" state, e.g., enumeration of addresses for memory access and PIN comparison operations.

State	Values of state variables	
	State[0] Done	State[1] Chk_enable
WAIT	0	0
CHECK	0	1
DONE	1	0

 © ANU 2026

State	Value of inputs				Next State	Output (at current state)	
	Start	Reset	Pin_no==data	address==15		*found*	*done*
WAIT	0	x	X	x	WAIT	0	0
WAIT	1	x	X	x	CHECK	0	0
CHECK	x	x	0	0	CHECK	0	0
CHECK	x	x	0	1	DONE	0	0
CHECK	x	x	1	x	DONE	0	0
DONE	x	0	0	x	DONE	0	1
DONE	x	0	1	x	DONE	1	1
DONE	x	1	X	x	WAIT	0	1

Table 1: State and transition/output table for the PIN Comparator.

Now to you: design the PIN comparator, starting from the template module declaration provided below.

```
module pincomp(
        input wire start,
        input wire [7:0] pin_no,
        input wire [7:0] data,
        input wire sysclk,
        input wire reset,
        output reg [3:0] addr,
        output reg done,
        output reg found);

// your work

endmodule
```

Module template for the PIN comparator (pincomp.v)

You can test your design with the test bench provided (*TB_comp_mem.v*). If you change the *pin_no* value in the test bench file you can simulate what happens when a matching PIN is found or otherwise.

Show your design / successful simulation result to the tutor when ready to receive your marks for this step.

 © ANU 2026

6. The Door Switch controller

The role of the door switch module is to detect when the door is opened after access is granted and then shut again. Upon this event, the main FSM controller should reset the system and wait for a new pin code to be entered.

The plan is to use one of the slide switches as the "door switch". The door is closed when the switch output is a logic level "1".

For this module, the code is provided (Code 6). The code exploits a *fast debouncer* and a *single pulse on transition* solution. This architecture is similar to designs we have seen in other labs.

A testbench file is provided if you think that would help you understand the way the module operates.

```
module doorshut(
     input wire reset,
     input wire closed_sw,
     input wire sysclk,
     output wire closed_fsm);

     wire pulse, ff_and;

     // heartbeat generator instantiation
     heartbeat #(16) hb(.sysclk(sysclk),.reset(reset),.pulse(pulse));

     reg ff_deb[2:0];
     reg ffout;

     always @(posedge sysclk) begin
     if (pulse) begin
        ff_deb[0]<=closed_sw; // ff chain
        ff_deb[1]<=ff_deb[0];
        ff_deb[2]<=ff_deb[1];
     end
        ffout<=ff_and; //ffout is the ff for single pulse on transition
     end

     assign ff_and=ff_deb[2]&ff_deb[1]&ff_deb[0];
     assign closed_fsm=ff_and&(~ffout);

endmodule
```

Verilog code for the door switch controller (doorshut.v)

 © ANU 2026

7. The hardware reset button

The implementation of the reset button is extremely basic: a simple synchroniser. See Code 7 below.

```
module reset_btn(
     input wire btn_in,
     input wire sysclk
     output reg reset_out);

     reg [1:0] ffsync;

     always @(posedge sysclk) begin
        ffsync[0]<=btn_in; //synchroniser. Why am I not using a debouncer for the reset?
        ffsync[1]<=ffsync[0];
        reset_out<=ffsync[1];
     end

endmodule
```

Code 7: Verilog code for the reset button synchroniser (reset_btn.v)

8. Putting it all together. The FSM controller *[11 marks]*

Now that you have all of the individual parts covered, it is time to put the system together. This will mean coding the FSM controller. This task is entirely up to you. A template module declaration is given below.

```
module fsm_controller(input wire sysclk,
     input wire ready,
     input wire done,
     input wire closed,
     input wire found,
     input wire reset_btn,
     output reg start,
     output reg reset
     output reg acc_gran);

     // your work

endmodule
```

Verilog template for the fsm_controller

Here are some recommended workflow steps:

1. Draw a timing diagram for the signals associated with the FSM controller. Start with an initial *reset* event and include the PIN number being entered leading to a *ready* input from the keypad controller, followed by *start, done, found, access granted* and *close* events for one operation cycle where access is successfully granted to one entrant and the door succeeds in closing after opening.
2. Identify suitable states for the FSM controller and sketch a state diagram

 © ANU 2026

3. Draw the next state table for the FSM controller
4. Draw the output table for the FSM controller
5. Write Verilog code for the controller. Refer from the template in Code 7 above for the names of the input/output signals.
6. Simulate your FSM controller design using the test bench provided
7. Write Verilog code for the full interconnected system. A top level file (*dooraccess.v*) has already been given. This instantiates all of the subcomponents but you may need to check that the instantiations work for your particular design. It might be useful for you to keep track of the names of the ports in the sub modules on a piece of paper in order to make sure that you have coded all of the interconnections.
8. Use Vivado to implement the design in hardware on the Zybo board. You will need to write the constraint file for proper I/O interfacing.
9. It may be interesting to review the "Elaborated Design" RTL schematic. This shows how the synthesiser interpreted your design structure. If all went well it should be a good match for Figure 1!
10. Check proper operation in hardware
11. Show your work to your demonstrator

Give your tutor an explanation of your FSM controller, using your state diagram, and next state / output tables. Also, demonstrate correct hardware operation of your system.

End of Lab 5

(Keep scrolling for Appendices)

Appendix 1: using a Basys3 board for this lab

The design can be attempted by students at home on a Basys3 board. The Basys board has a greater number of LEDs, allowing the display of more interesting LED patterns, e.g., for pin entry count and "access granted". If you adopt this, you will need to adjust your LED bus sizes accordingly.

 © ANU 2026